Embedded Data Structure
(Lecture Note 7)

Linked List (1)

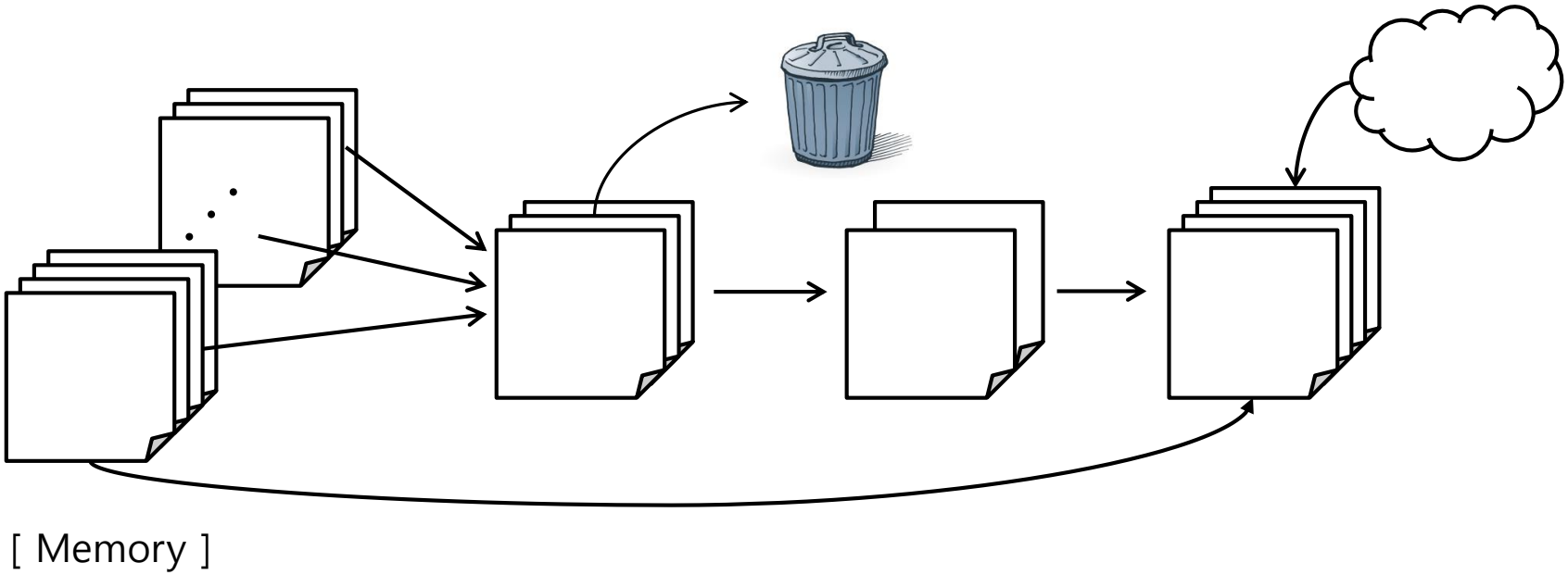
2025. 3. 25

Prof. Wonjun Kim

School of Electrical and Electronics Engineering

Introduction (1/2)

- **Linked list (연결 리스트)**
  - **Easy way to access the memory (dynamic linking)**



Memory handling with the linked list

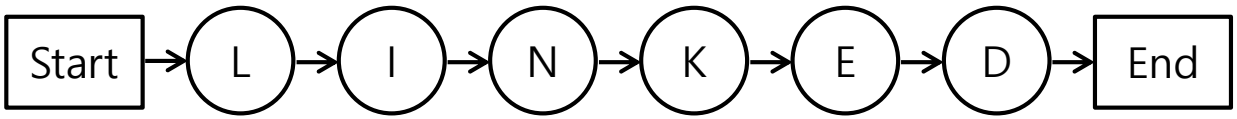
➔ **Linked list makes possible data structure to be flexible**

Introduction (2/2)

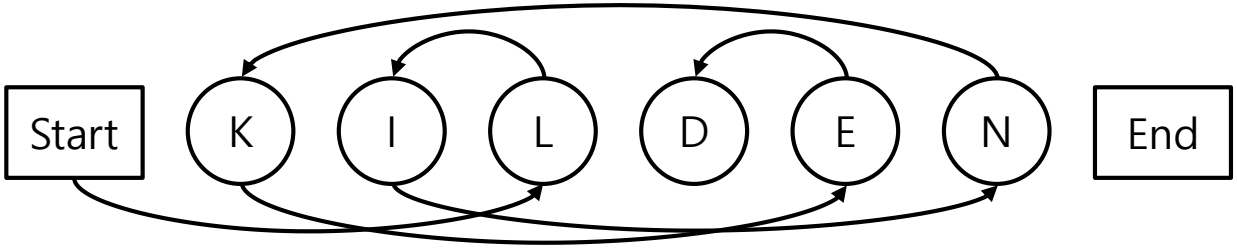
▪ The heart of the data structure

- **Linked list is composed of "node" and "link"**

  - Node : store the data / Link : connect nodes



[Linked List (conceptual flow)]



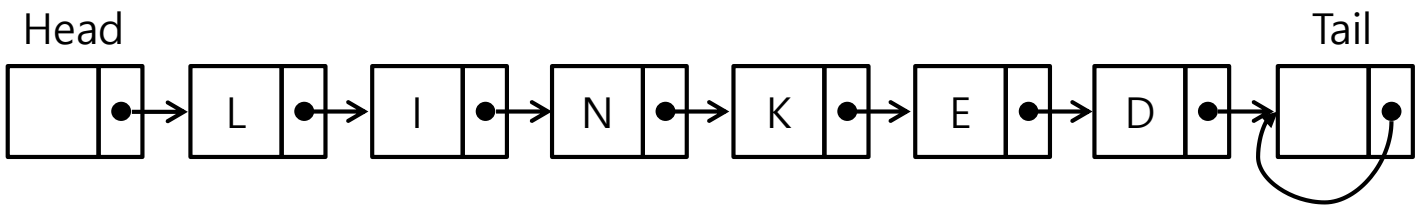
[Real relationship in the memory]

☀ Dynamic data structure (cf. array)

What are advantages of the dynamic data structure ?

Simple Linked List (1/13)

- **Node and link (indicating the next node)**
  - **It is useful for the re-arrangement of the given data**



[Simple linked list (example)]

- Head : the entrance of the linked list (global variable)

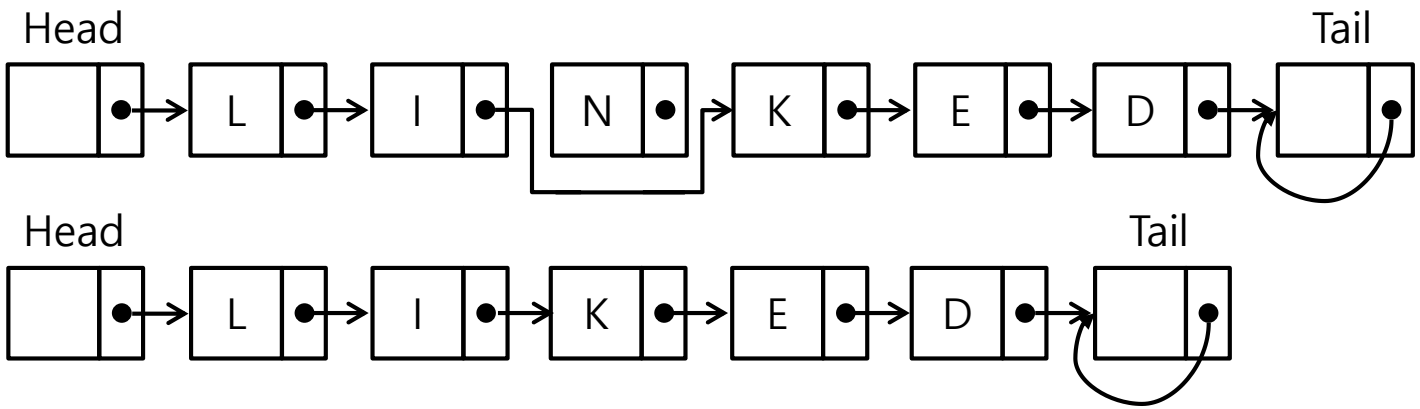
  ➔ Why is the head defined as "global variable" ?

- Tail : the exit of the linked list (last node should indicate something !)

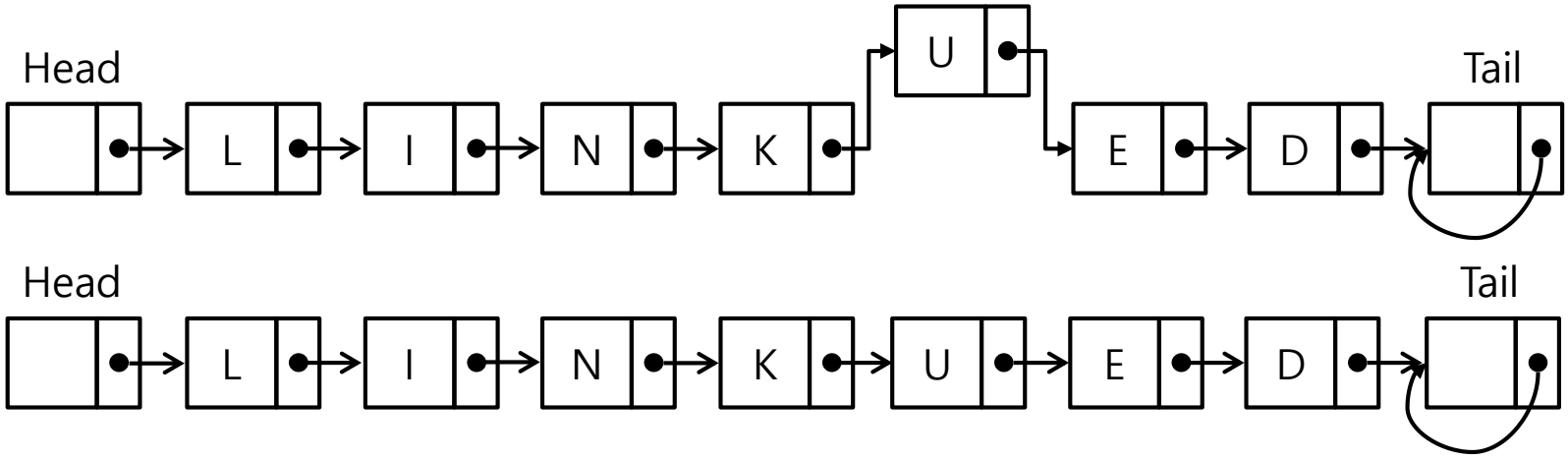
※ Other way : head → pointer, tail → NULL

Simple Linked List (2/13)

- **Examples of insertion and delete**



[Remove "N" from linked list (memory free)]



[Insert "U" in the linked list]

Simple Linked List (3/13)

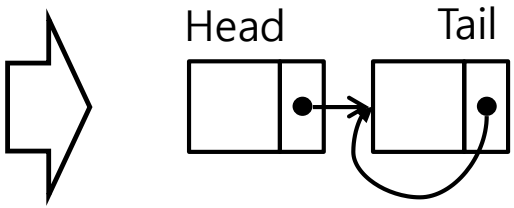
- **Linked list based on C**
  - **Definition of the node**

```
typedef struct _node
{
    int key;                 → Store a value
    struct _node *next;      → Indicating the next position
}node;
```

  - **Initialization of the linked list**

```
node *head, *tail;

void init_list()
{
    head = (node *)calloc(1, sizeof(node));
    tail = (node *)calloc(1, sizeof(node));
    head->next = tail;
    tail->next = tail;
}
```



Simple Linked List (4/13)

KU KONKUK UNIVERSITY

▪ Linked list based on C – cont'd

- **Insert a new node after the current node**

```
node *insert_after(int k, node *t)
{
    node *temp;
    temp = (node *)calloc(1, sizeof(node));
    temp->key = k;
    temp->next = t->next;
    t->next = temp;

    return temp;
}
```

What is the difference ?

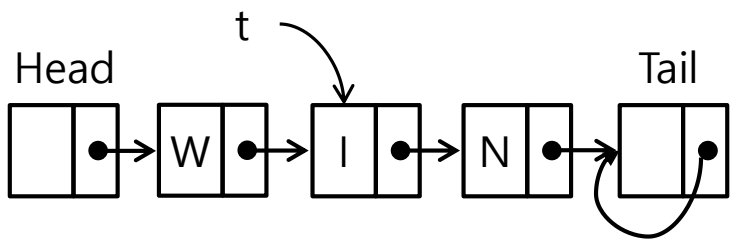
※ We do not know the front node of the current one in the simple linked list

➔ This is because the node structure only contains the next position !

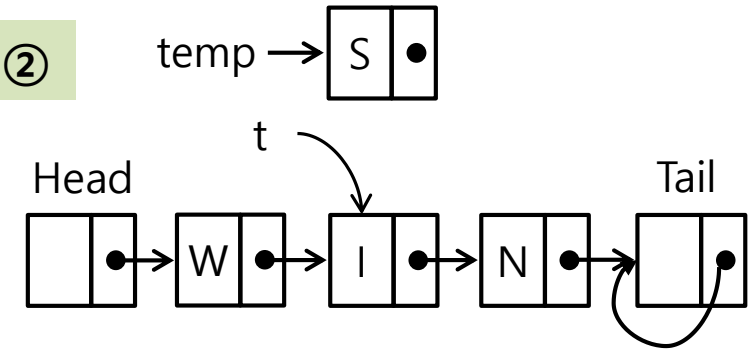
Simple Linked List (5/13)

- **Linked list based on C – cont'd**
  - **Pictorial example : insertion**

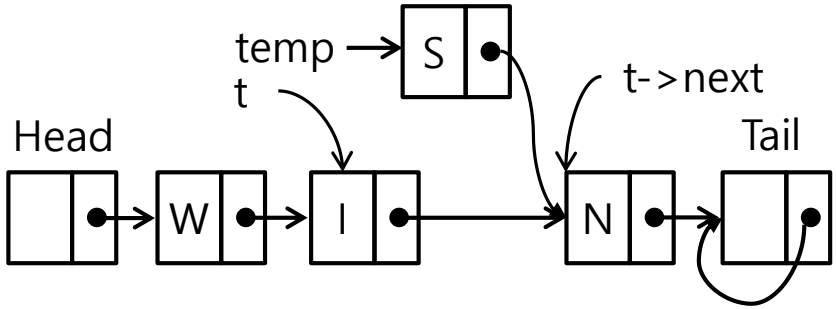
①



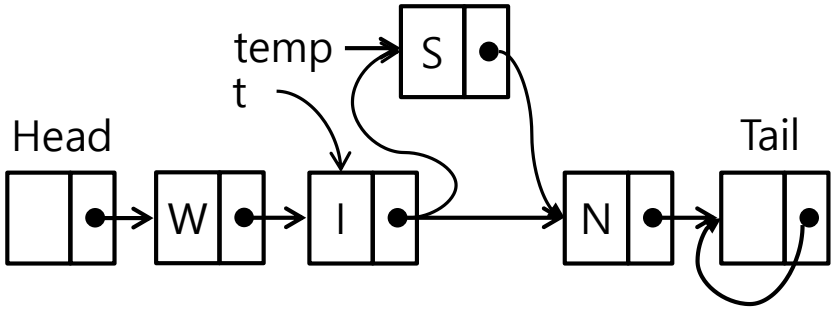
②



③ **temp->next = t->next**



④ **t->next = temp**



KU KONKUK UNIVERSITY

▪ Linked list based on C – cont'd

- **Delete the next node**

```
int delete_next(node *t)
{
    node *temp;
    if(t->next == tail)
        return 0;
    temp = t->next;
    t->next = t->next->next;
    free(temp);

    return 1;
}
```

→ We want to delete the next node of t

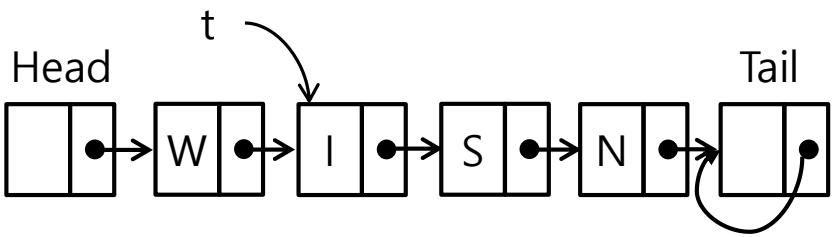
→ Check the type of the current node

※ We need to check whether the current node is tail or not

➔ This is due to the stability of the given data structure

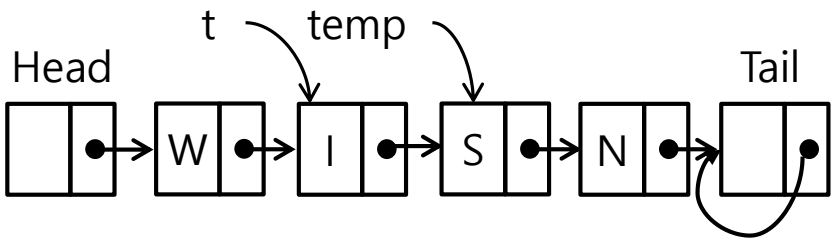
- **Linked list based on C – cont'd**
  - **Pictorial example : delete**

①



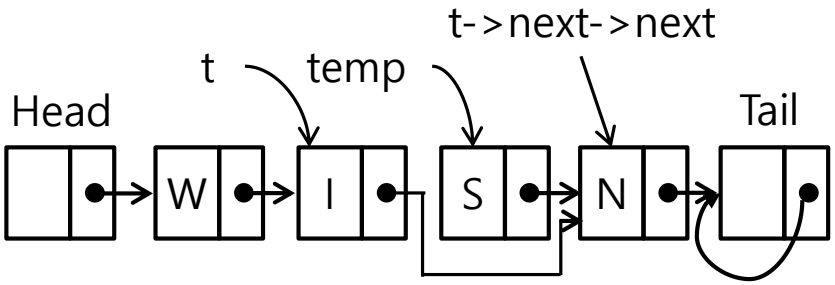
②

temp = t->next



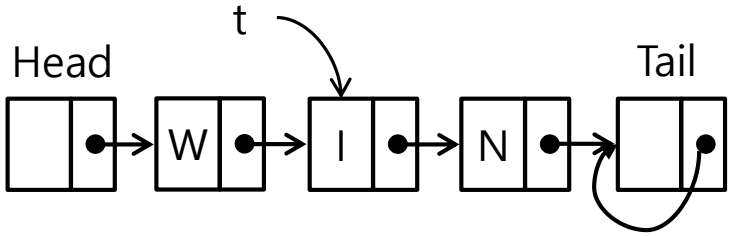
③

t->next = t->next->next



④

free(temp)



Simple Linked List (8/13)

- **Linked list based on C – cont'd**
  - **Find the node containing a given key value**

```
node *find_node(int k)
{
    node *temp;

    temp = head->next;
    while(temp->key != k && temp != tail)
        temp = temp->next;

    return temp;
}
```

→ Check the both cases (find or not) !

※ Find a given key value → return the corresponding node

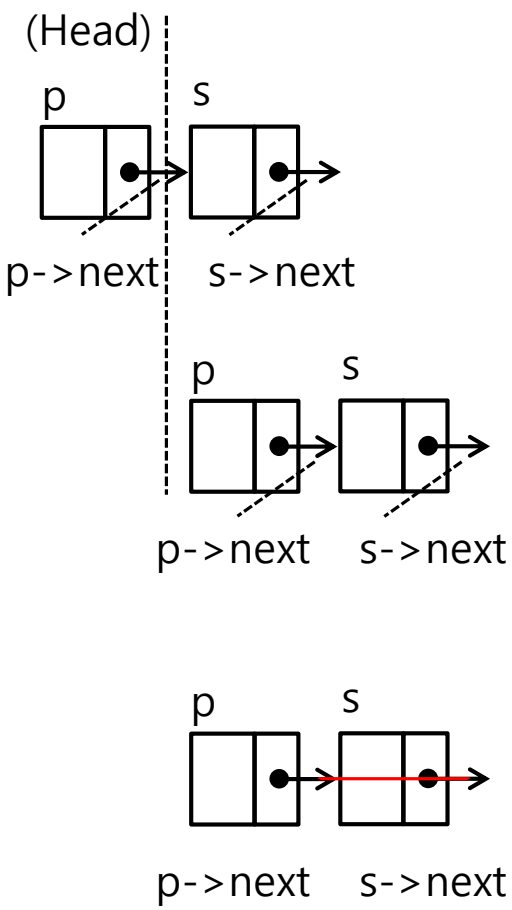
Fail to find a given key value → return the tail node

KU KONKUK UNIVERSITY

▪ Linked list based on C – cont'd

- **Delete the node containing a given key value**

```
int delete_node(int k)
{
    node *s, *p;  ⟶ Previous node of "s"
    p = head;
    s = p->next;
    while(s->key != k && s != tail) {
        p = p->next;
        s = p->next;
    }
    if( s != tail) {
        p->next = s->next;
        free(s);
        return 1;
    }
    else return 0;
}
```



Simple Linked List (10/13)

- **Linked list based on C – cont'd**
  - **Insert the node containing a given key value**

```
node *insert_node(int t, int k)
{
    node *s, *p, *r;
    p = head;
    s = p->next;
    while(s->key != k && s != tail) {
        p = p->next;
        s = p->next;
    }
    if( s != tail) {
        r = (node *)calloc(1, sizeof(node));
        r->key = t;
        p->next = r;
        r->next = s;
    }
    return p->next;
}
```

Insert "t" before "k"

Think about the flow

Simple Linked List (11/13)

KU KONKUK UNIVERSITY

- **Linked list based on C – cont'd**
  - **Ordered sorting in the linked list (dynamic linking)**

```
node *ordered_insert(int k)
{
    node *s, *p, *r;
    p = head;
    s = p->next;
    while(s->key <= k && s != tail) {
        p = p->next;
        s = p->next;
    }
    r = (node *)calloc(1, sizeof(node));
    r->key = k;
    p->next = r;
    r->next = s;
    return r;
}
```

For example,

Current list : 1→3→4→6
ordered_insert(5), then

Simple Linked List (12/13)

KU KONKUK UNIVERSITY

- **Linked list based on C – cont'd**
  - **Delete all the linked list**

```
Node *delete_all()
{
    node *s;
    node *t;
    t = head->next;
    while(t != tail)
    {
        s = t;
        t = t->next;
        free(s);
    }
    head->next = tail;
    return head;
}
```

What is the difference with

head->next = tail ?

Print out the current list !

```
void print_list(node *t)
{
    while(t != tail) {
        printf("%-8d", t->key);
        t = t->next;
    }
}
```

Simple Linked List (13/13)

- **Linked list based on C – cont'd**
  - **Practice : final check your own list !**

```
void main()
{
    node *t;

    init_list();
    ordered_insert(9);
    ordered_insert(3);
    ordered_insert(5);
    ordered_insert(1);
    ordered_insert(7);

}
```

1. Print out initial list
2. Find the node containing "5"
3. Insert 6 after 5 and

   Print out the current list
4. Delete node 5 and print out the result
5. Delete all the node

Summary

- Linked list for dynamic handling the memory space
  - Think about the role of pointer !
- Draw your list for better understanding